

Scaling, Post-Training & Evaluation

ECE 247B · LECTURE 2

Shreyas Rajesh

Spring 2026

From compute-optimal base
models to useful assistants.

Today's Story

01

Scaling Laws

Pick up where Lecture 1 ended: why GPT-style pretraining became a scaling problem.

02

Chinchilla

Ask how to split fixed compute between parameters and training tokens.

03

ChatGPT

Explain why a strong base model is still not an assistant.

04

Post-Training

SFT, preference data, reward models, PPO, DPO, and modern recipes.

05

Reasoning Models

Move from preference rewards to verifiable rewards and test-time compute.

06

Evaluation

Once behavior changes, how do we know the model improved?

Lecture 1 built the base-model recipe. Lecture 2 asks how to allocate scale, turn a base model into an assistant, and then understand why reasoning became the next post-training frontier.

Part I: Scaling Laws

Where Lecture 1 Ended

The recipe

- Tokenize text.
- Train a decoder-only transformer.
- Optimize next-token cross-entropy.
- Generate autoregressively.

$$\mathcal{L} = - \sum_t \log p_\theta(x_t \mid x_{<t})$$

The empirical surprise

The same simple objective kept improving as researchers increased model size, data, and compute.

The natural question became less “which objective?” and more “how far can this recipe scale?”

Today starts by making that scaling question precise.

The Scaling-Laws Question

Model size

How does loss change as the number of parameters increases?

N
parameters

Dataset size

How does loss change as we train on more tokens?

D
tokens

Compute

How does loss change as we spend more FLOPs?

C
training compute

If loss follows predictable curves, then scale becomes an engineering decision, not just a scientific gamble.

Kaplan Scaling Laws

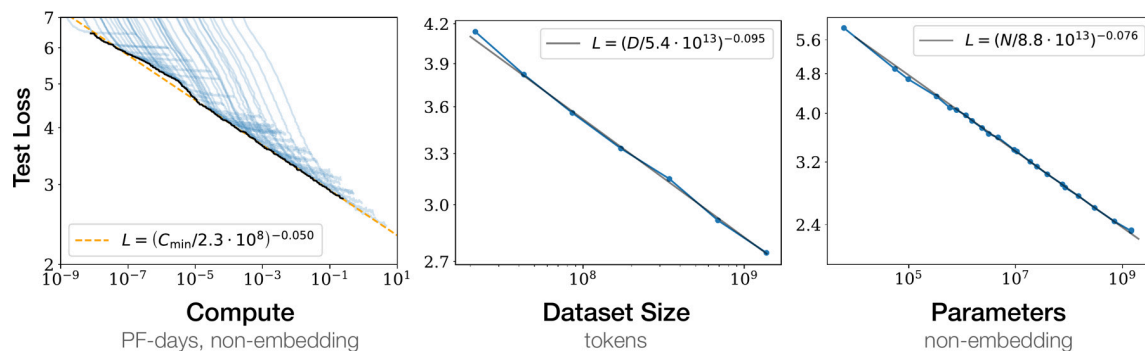


Figure source: Kaplan et al., “Scaling Laws for Neural Language Models,” 2020, arXiv:2001.08361 source, Figure 1.

The empirical punchline

- Loss decreases smoothly with compute, data, and parameters.
- The curves look close to power laws over many orders of magnitude.
- This gave GPT-3-era labs a reason to believe scale was a reliable lever.

Scaling laws made pretraining feel forecastable.

Kaplan: Shape Mostly Washes Out

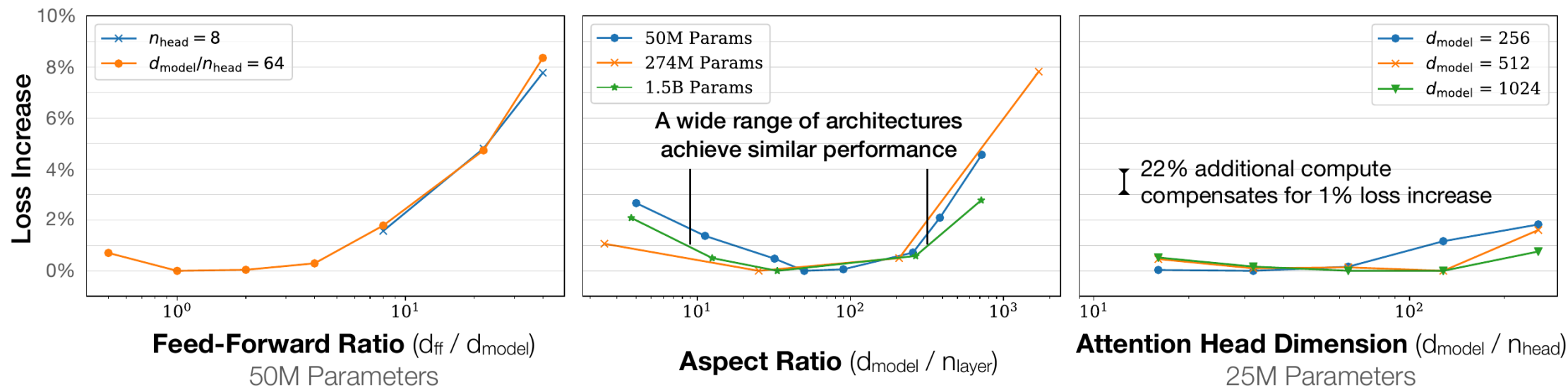


Figure source: Kaplan et al., "Scaling Laws for Neural Language Models," 2020, arXiv:2001.08361 source, Figure 2.

Fixed size

Hold non-embedding parameter count approximately fixed.

Vary shape

Change feed-forward ratio, depth/width aspect ratio, or attention head dimension.

Small penalty

A broad range of architectures lands near the same loss; extremes hurt more.

This is part of why scaling laws felt powerful: after reasonable architecture choices, total scale dominated many smaller design knobs.

Kaplan Takeaway: Scale the Whole Recipe

The message was not “invent a clever new loss.” It was: the simple decoder-only next-token recipe keeps improving as the axes scale.

More parameters

More capacity can represent more structure.

More tokens

More data provides more learning signal.

More compute

Compute lets us scale parameters and data together.

GPT-3-era lesson

Large models trained on large datasets keep improving smoothly.

Open question

Given a fixed compute budget, how much should we spend on parameters versus tokens?

Kaplan's Compute-Efficient Prescription

Fitted loss exponents

α_N	0.076	loss vs. non-embedding parameters
α_D	0.095	loss vs. training tokens
$\alpha_C^{\min}$	0.050	loss vs. compute-efficient training compute

Optimal allocation from Kaplan

$$N_{\text{opt}} = 1.3 \times 10^9 C_{\min}^{0.73}$$

$$D_{\text{opt}} = 2 \times 10^{10} C_{\min}^{0.27}$$

Compute-efficient exponents: $p_N = 0.73$, $p_D = 0.27$.

In their fit, most additional compute goes into larger models; tokens grow much more slowly.

This is the key Kaplan-to-Chinchilla handoff: Kaplan predicts very large, relatively undertrained models; Chinchilla asks whether that allocation was too parameter-heavy.

Source: Kaplan et al., "Scaling Laws for Neural Language Models," 2020, arXiv:2001.08361, Tables 5-6. Here $C_{\min}$ uses the paper's compute units.

GPT-3: Few-Shot Prompting

Prompt as a tiny dataset

- 1 Translate English to French:
- 2 sea otter => loutre de mer
- 3 cheese => **fromage**
- 4 banana => ?

The model sees examples, then completes the pattern.

What made this different

- The examples are part of the input sequence.
- The parameters are not updated for the task.
- The same model can switch tasks when the prompt changes.

Few-shot no longer meant “train a small model on a few examples.” It meant “condition a huge pretrained model on a few examples.”

Source: Brown et al., “Language Models are Few-Shot Learners,” 2020, adapted from the GPT-3 few-shot translation setup.

Why In-Context Learning Was Strange

Classical supervised ML

Examples live in a training set. An optimizer uses them to change model parameters.

In-context learning

Examples live in the prompt. The model performs a new task during one forward pass.

The conceptual shift

A task can be specified by natural language and demonstrations, not by a new training loop.

What is fixed?

The weights. No gradient step is taken when the prompt contains examples.

What changes?

The context. The examples become evidence that conditions the next-token distribution.

This is why GPT-3 mattered: scale did not just lower loss; it changed how people interacted with models.

Source: Brown et al., "Language Models are Few-Shot Learners," 2020.

The Compute Allocation Problem

Training compute is finite

For dense decoder transformers, a useful approximation is:

$$C \approx 6ND$$

Here N is parameters and D is training tokens.

The tradeoff

- If N is too large, the model is undertrained.
- If D is too large for a tiny model, capacity bottlenecks.
- Compute-optimal training must balance both.

This is the bridge from Kaplan to Chinchilla.

Part II: Chinchilla

Chinchilla: Read the IsoFLOP Plot

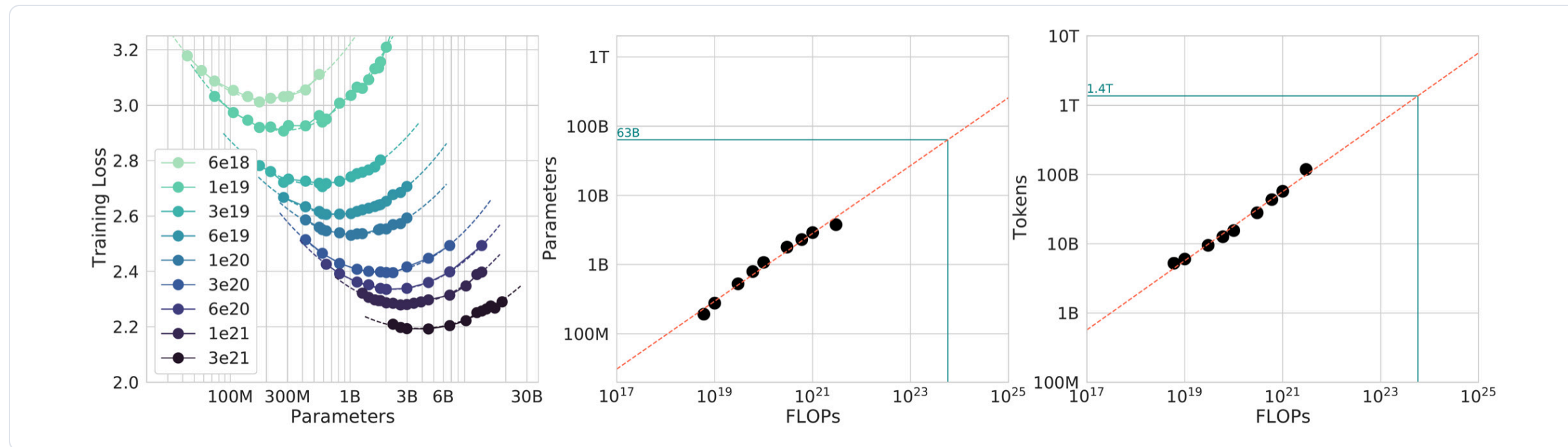


Figure source: Hoffmann et al., "Training Compute-Optimal Large Language Models," 2022, arXiv:2203.15556 source, IsoFLOP curves figure.

1. Fix compute

Each curve holds the FLOP budget constant.

2. Sweep model size

Larger models leave fewer tokens in the same budget.

3. Find the valley

The minimum loss gives the compute-optimal split.

Chinchilla Rule of Thumb

Compute-optimal balance

$$C \approx 6ND$$

$$D/N \approx 20$$

Roughly: train on about 20 tokens per parameter.

Concrete example

Model	Parameters	Tokens
GPT-3	175B	~300B
Chinchilla	70B	~1.4T

The headline was not just “bigger model.” It was “more data for the compute budget.”

Sources: Brown et al., “Language Models are Few-Shot Learners,” 2020; Hoffmann et al., “Training Compute-Optimal Large Language Models,” 2022.

Beyond Chinchilla: Keep Training?

What Chinchilla really says

For a fixed training-compute budget, choose a model size and token count that minimize loss.

$$D/N \approx 20 \quad \text{at the Chinchilla optimum}$$

It does not say that additional tokens stop being useful.

Why modern models overtrain

- Inference cost matters: smaller models are cheaper to serve.
- Training data is often cheaper than serving a much larger model forever.
- High-quality extra tokens can still improve downstream performance.

Model family	Params	Pretraining tokens	Tokens / param
Chinchilla rule	70B	~1.4T	~20
Llama 2	7B / 13B / 70B	2T	~286 / ~154 / ~29
Llama 3	8B / 70B	15T+	~1,875 / ~214
Liquid LFM2	350M / 700M / 1.2B	10T	~28k / ~14k / ~8.3k

Sources: Hoffmann et al., “Training Compute-Optimal Large Language Models,” 2022; Touvron et al., “Llama 2: Open Foundation and Fine-Tuned Chat Models,” 2023; Meta, “Introducing Meta Llama 3,” 2024; Liquid AI, “Introducing LFM2,” 2025.

What Chinchilla Changed

Before

Many large models were trained with too few tokens for their size.

After

Compute-optimal pretraining treats data allocation as part of the recipe.

Consequence

Scaling means coordinating model size, token count, data quality, and systems constraints.

Now we know how to train stronger base models. But a compute-optimal base model is still a completion model, not necessarily an assistant.

What Does “Data” Mean at Pretraining Scale?

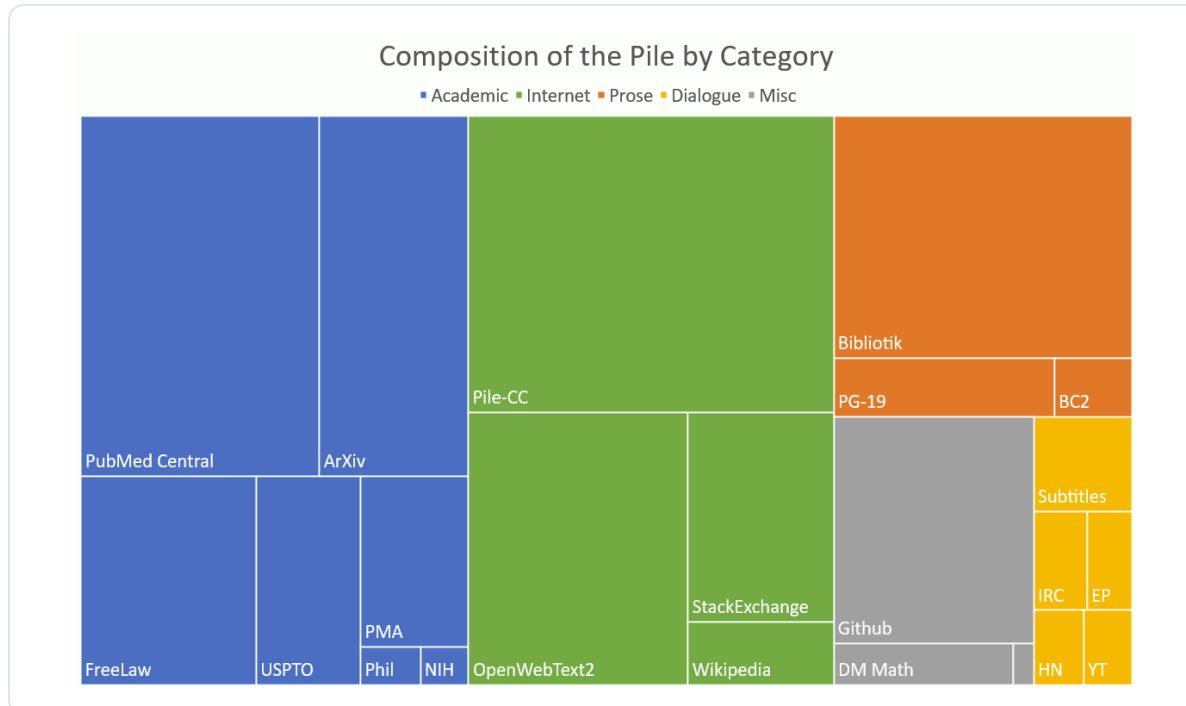


Figure source: Gao et al., “The Pile: An 800GB Dataset of Diverse Text for Language Modeling,” 2020/2021, arXiv:2101.00027 source, Figure 1.

The Pile as a concrete example

- 825 GiB English corpus built from 22 components.
- Mixture includes filtered web text, books, academic papers, code, law, forums, subtitles, and reference text.
- Components are sampled with different epoch weights; the figure shows effective size.
- The data mixture is a modeling choice: it changes what domains the base model sees often enough to learn well.

Scaling laws talk about token counts; real pretraining is also about which tokens.

The LLM Training Pipeline

A modern LLM is not produced by one training run. The model is moved through stages that change what the distribution is supposed to teach.

01

Pretraining

Next-token prediction on broad text, code, and math data. This builds the base model: knowledge, language fluency, and many latent capabilities.

02

Midtraining

Continued training on targeted mixtures: higher-quality data, code/math, domain data, or long-context data. The objective often stays language modeling; the data distribution changes.

03

Post-training

Instruction tuning, preference optimization, RLHF/DPO, and verifier-based RL. This turns a capable base model into a useful interactive system.

Lecture 2: finish the pretraining story, then study how post-training changes behavior without changing the basic transformer skeleton.

Sources: Touvron et al., “Llama 2: Open Foundation and Fine-Tuned Chat Models,” 2023; Meta, “Introducing Meta Llama 3,” 2024; DeepSeek-AI, “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,” 2025.

What Pretraining Gives You

Powerful base model

- Broad language, facts, code, and world knowledge.
- Strong next-token prediction.
- Ability to continue prompts in many styles.

Missing assistant behavior

- Directly follow user intent.
- Use conversational formatting.
- Prefer helpful over merely plausible continuations.
- Refuse or redirect unsafe requests.

This is the base-model side of the story: broad competence from next-token prediction, but not yet reliable assistant behavior.

Pretraining Is Massive Multitask Learning

<u>Task</u>	<u>Example sentence in pre-training that would teach that task</u>
<i>Grammar</i>	In my free time, I like to { <u>code</u> , banana}
<i>Lexical semantics</i>	I went to the store to buy papaya, dragon fruit, and { <u>durian</u> , squirrel}
<i>World knowledge</i>	The capital of Azerbaijan is { <u>Baku</u> , London}
<i>Sentiment analysis</i>	Movie review: I was engaged and on the edge of my seat the whole time. The movie was { <u>good</u> , bad}
<i>Translation</i>	The word for “pretty” in Spanish is { <u>bonita</u> , hola}
<i>Spatial reasoning</i>	Iroh went into the kitchen to make tea. Standing next to Iroh, Zuko pondered his destiny. Zuko left the { <u>kitchen</u> , store}
<i>Math question</i>	Arithmetic exam answer key: $3 + 8 + 4 =$ { <u>15</u> , 11}

[millions more]

Extreme multi-task learning!

Figure source: Jason Wei, “Intuitions on language models,” Stanford CS25 guest lecture, 2024, slide 6.

A single next-token objective hides many supervised tasks inside the text stream: grammar, semantics, world knowledge, translation, sentiment, reasoning, math, code, and millions more.

Before ChatGPT: Scale Looked Like Capabilities

May 2020

GPT-3 few-shot

Task examples in the prompt can specify new behavior without gradient updates.

Jan 2022

Chain-of-thought

Reasoning improves when few-shot examples include intermediate steps.

Mar 2022

InstructGPT

RLHF becomes the canonical recipe for instruction-following behavior.

May 2022

Zero-shot CoT

A simple prompt such as “Let’s think step by step” can elicit reasoning without demonstrations.

Jun 2022

Emergent abilities

Some benchmark curves appear flat until a large enough model scale.

Nov 2022

ChatGPT

These ingredients become visible to the public as a conversational product.

Before ChatGPT launched, the field had already seen a pattern: scale and prompting could reveal surprising behaviors, especially around reasoning.

Sources: Brown et al., “Language Models are Few-Shot Learners,” 2020; Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” 2022; Ouyang et al., “Training Language Models to Follow Instructions with Human Feedback,” 2022; Kojima et al., “Large Language Models are Zero-Shot Reasoners,” 2022; Wei et al., “Emergent Abilities of Large Language Models,” 2022.

Chain-of-Thought Prompting

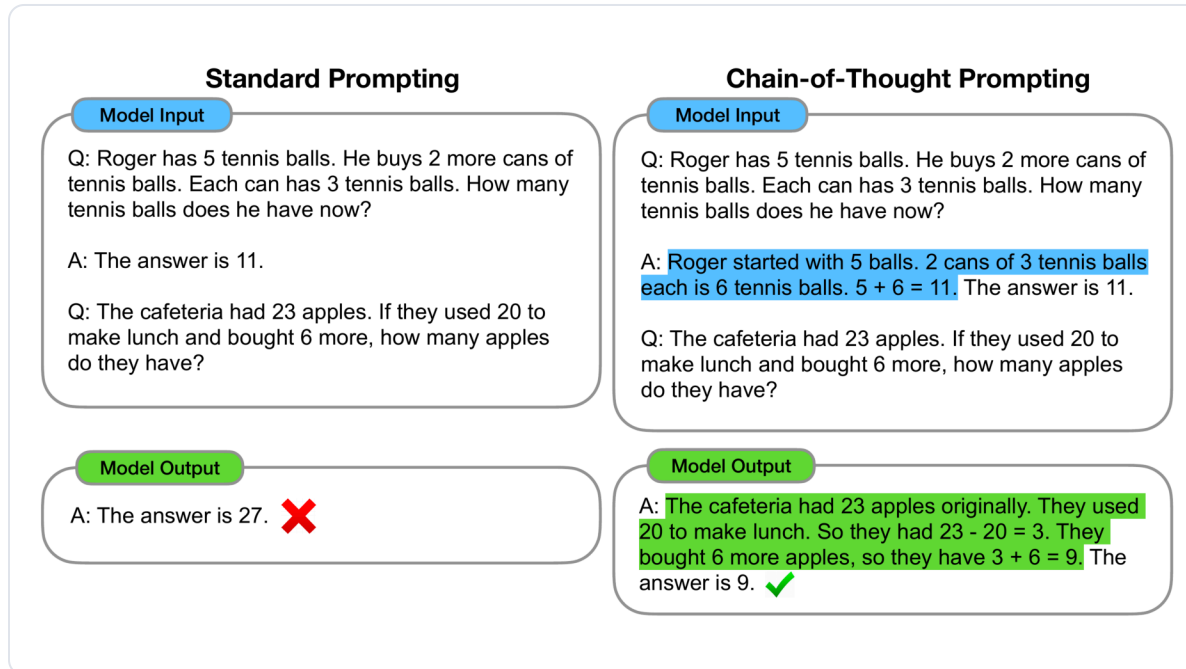


Figure source: Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," 2022, arXiv:2201.11903 source, Figure 1.

The new move

- Keep the same pretrained model.
- Change the demonstrations to include intermediate reasoning.
- Let the model spend extra tokens before the final answer.

Prompting starts to look like controlling the computation the model performs at inference time.

Zero-Shot CoT: “Let’s Think Step by Step”

Few-shot, standard		Few-shot CoT	
In	Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now? A: The answer is 11. per task example × N Q: While shopping for music online, Janet bought 6 country albums and 2 pop albums. Each album came with a lyric sheet and had 9 songs. How many songs did Janet buy total? A:	Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now? A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11. per task chain of thought example × N Q: While shopping for music online, Janet bought 6 country albums and 2 pop albums. Each album came with a lyric sheet and had 9 songs. How many songs did Janet buy total? A:	
Out	The answer is xxx	Janet bought 6 country albums and 2 pop albums. That is a total of 8 albums. Each album has 9 songs. So 8 * 9 = 72. The answer is 72	
Zero-shot, standard		Zero-shot CoT	
In	Q: While shopping for music online, Janet bought 6 country albums and 2 pop albums. Each album came with a lyric sheet and had 9 songs. How many songs did Janet buy total? A:	Q: While shopping for music online, Janet bought 6 country albums and 2 pop albums. Each album came with a lyric sheet and had 9 songs. How many songs did Janet buy total? A: Let's think step by step.	
Out	The answer is xxx	Janet bought 6 country albums and 2 pop albums. That is a total of 8 albums. Each album has 9 songs. So 8 * 9 = 72. The answer is 72	

Figure source: Kojima, Gu, Reid, Matsuo, and Iwasawa, “Large Language Models are Zero-Shot Reasoners,” 2022, arXiv:2205.11916 source, conceptual comparison figure.

Few-shot CoT

Worked reasoning examples are included in the prompt.

Zero-shot CoT

A task-agnostic trigger asks the model to reason before answering.

Why exciting?

The prompt seemed to reveal a latent behavior, not train a new skill.

Emergent Abilities: The Excitement

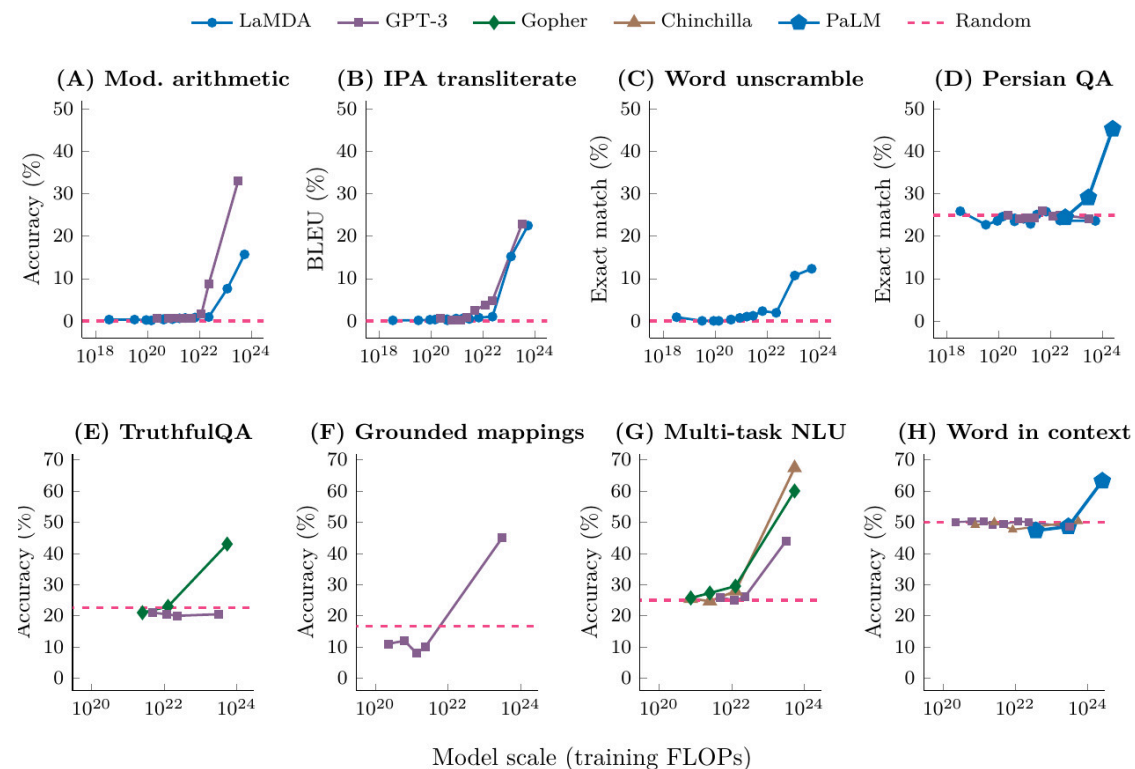


Figure source: Wei et al., “Emergent Abilities of Large Language Models,” 2022, arXiv:2206.07682 source, Figure 2.

Some downstream metrics looked near-random at smaller scales, then jumped above the random baseline at larger scales. Caveat: “emergence” can depend on the metric and evaluation setup.

Prompting Itself Looked Scale-Dependent

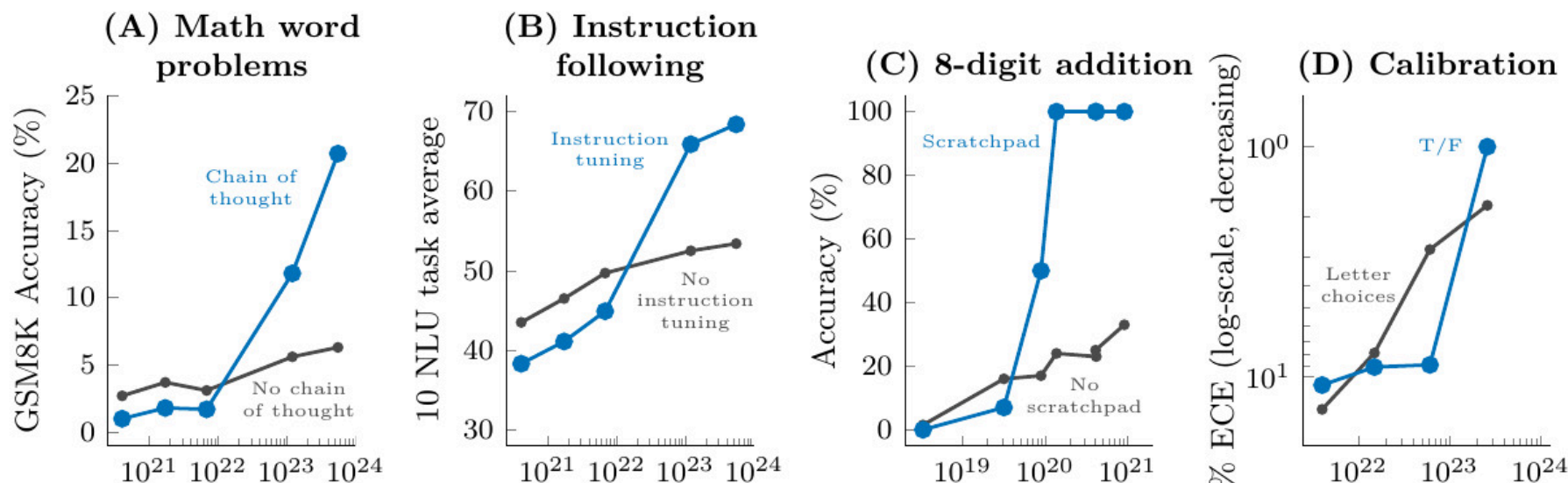


Figure source: Wei et al., “Emergent Abilities of Large Language Models,” 2022, arXiv:2206.07682 source, Figure 3.

CoT

Intermediate reasoning helps only once the model is large enough.

Scratchpads

Extra intermediate tokens can change what computation is available.

Bridge

If prompting can unlock behavior, post-training can try to make that behavior reliable.

Part III: From Base Models to ChatGPT

ChatGPT: The Behavior Shift

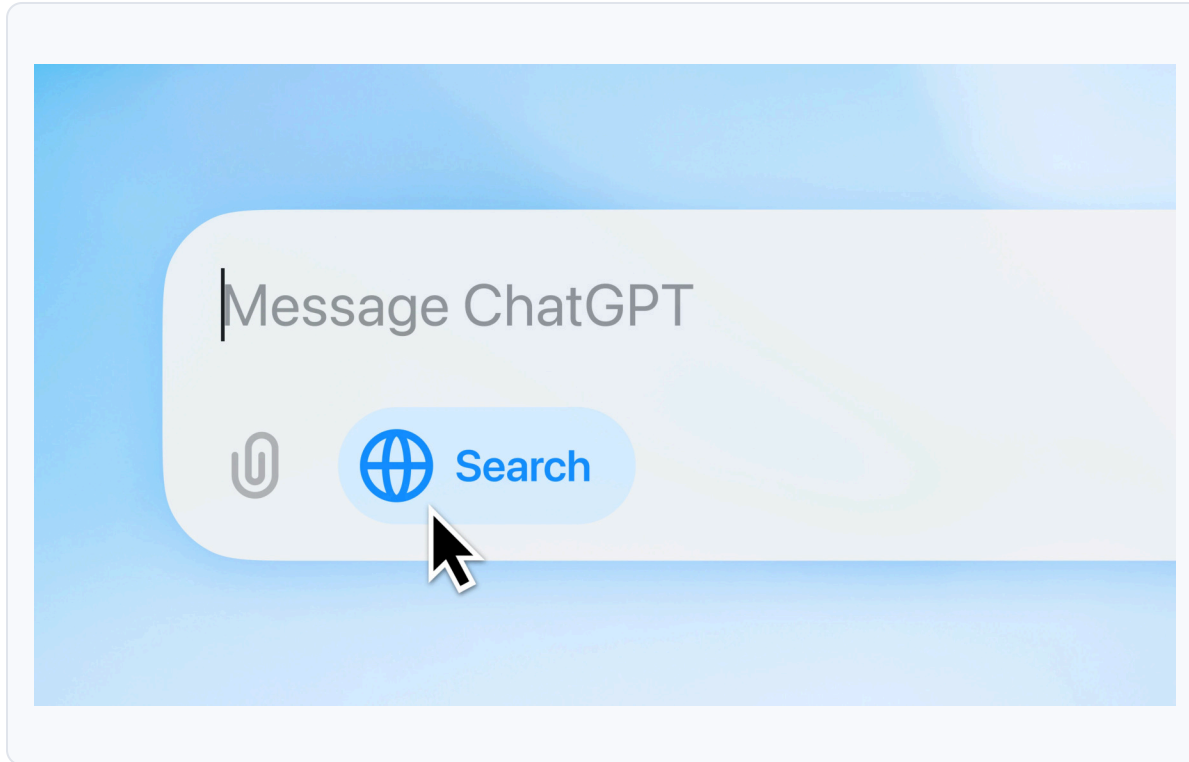


Figure source: Nathan Lambert, SALA 2026 RLHF/post-training deck, ChatGPT slide.

Same foundation, different behavior

- Base models continue text.
- Chat models follow instructions in a turn-taking interface.
- The behavior shift comes from post-training data and optimization.

ChatGPT did not make next-token prediction irrelevant. It changed which next tokens were rewarded.

The Three-Stage Recipe: Start with SFT

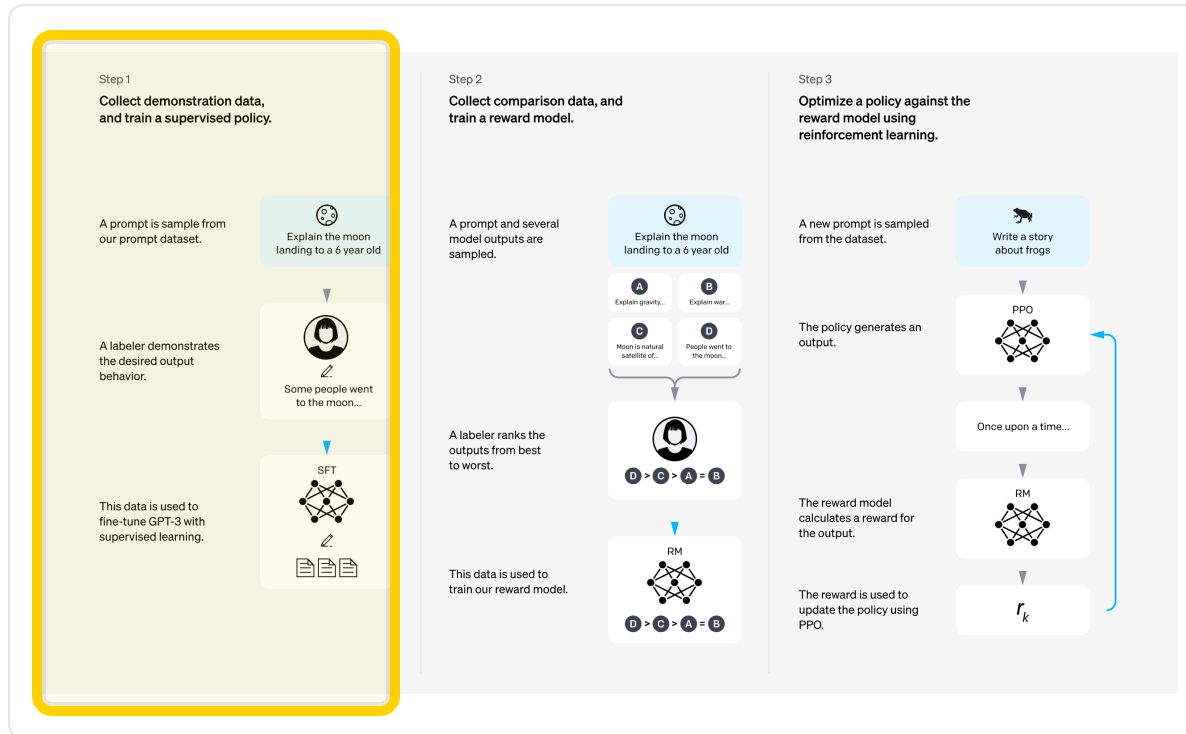


Figure source: Nathan Lambert SALA 2026 deck, adapted from Ouyang et al., "Training language models to follow instructions with human feedback," 2022.

1. Supervised fine-tuning

Teach the base model the assistant data format by imitating demonstrations.

2. Reward modeling

Use pairwise human comparisons to learn which responses are preferred.

3. Policy optimization

Optimize the assistant for high reward while staying close to the SFT model.

The first move is still supervised learning: collect demonstrations, then train the model to imitate the assistant side of the exchange.

Stage 1: Instruction-Tuning Data

A supervised chat row

System: You are a helpful assistant.

User: Explain dropout in one short paragraph.

Assistant: Dropout randomly disables a fraction of hidden units during training, which prevents co-adaptation and acts like regularization.

What changes from pretraining?

- The input is now an instruction or dialogue context.
- The target is a desired assistant response.
- Data includes formatting, style, safety, refusal, and task-following behavior.
- The model is trained to produce the assistant side of the conversation.

Sources: Wei et al., “Finetuned Language Models Are Zero-Shot Learners,” 2021; Ouyang et al., “Training Language Models to Follow Instructions with Human Feedback,” 2022; Chung et al., “Scaling Instruction-Finetuned Language Models,” 2022.

FLAN: What the Rows Looked Like

Email subject generation

Instruction: Write a subject line for an email about finalizing a signed document.

document-signing email -> "Ronald Chisholm LOI"

Topic classification

Instruction: What is this text about? Choose from World, Sports, Business, Science/Tech.

retail acquisition article -> Business

Summarization

Instruction: Write highlights for this article.

travel article about The Hague -> short highlights

Data-to-text

Instruction: Write a sentence that includes the restaurant facts.

name=Aromi, food=English, area=city centre -> restaurant sentence

FLAN-style instruction tuning repackaged many supervised NLP tasks as natural-language prompt to target pairs. The model learns “how to be instructed” across tasks.

Sources: Tatsu Hashimoto, Stanford CS336 Lecture 15, “Alignment - SFT/RLHF,” 2025, FLAN examples slide; Wei et al., “Finetuned Language Models Are Zero-Shot Learners,” 2021; Longpre et al., “The Flan Collection,” 2023.

Alpaca Data: Synthetic Instruction Rows

General advice

Instruction: Give three tips for staying healthy.

output: eat well; exercise; sleep consistently

Definition

Instruction: What does “algorithm” mean?

output: a step-by-step procedure for solving a problem

Code generation

Instruction: Find the average number in a list.

output: `return sum(nums) / len(nums)`

How it was made

Alpaca fine-tuned LLaMA 7B on 52K instruction-following demonstrations generated from text-davinci-003.

175 seed tasks -> synthetic instruction data

Alpaca made the post-training data recipe feel reproducible: use a strong teacher model to synthesize many instruction-output pairs, then do ordinary supervised fine-tuning.

Sources: Tatsu Hashimoto, Stanford CS336 Lecture 15, “Alignment - SFT/RLHF,” 2025, Alpaca examples slide; Taori et al., “Alpaca: A Strong, Replicable Instruction-Following Model,” Stanford CRFM blog, 2023; Wang et al., “Self-Instruct,” 2022.

Stage 1: The SFT Objective

Same next-token loss

Given a prompt or chat context x and a demonstration answer y^* , maximize the likelihood of the demonstration tokens.

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_t \log \pi_{\theta}(y_t^* \mid x, y_{<t}^*)$$

Different data distribution

- Pretraining: predict arbitrary next tokens in raw text.
- SFT: predict assistant tokens conditioned on an instruction.
- Teacher forcing reveals previous gold answer tokens during training.
- Often the loss is masked so prompt tokens condition the model but are not the target.

Instruction tuning does not change the basic language-modeling machinery. It changes what distribution the model is asked to imitate.

What SFT Can and Cannot Do

Good at

Teaching chat format, tone, style, instruction following, refusals, and common response patterns.

Limited by

It imitates written demonstrations and does not directly use comparisons between better and worse answers.

Next question

If judging is easier than writing ideal answers, can we train from preferences?

Sources: Ouyang et al., “Training Language Models to Follow Instructions with Human Feedback,” 2022; Stiennon et al., “Learning to summarize from human feedback,” 2020.

Why Not Just SFT Good Answers?

We can, and we do

SFT over acceptable completions is the first major post-training step.

$$\mathcal{L}_{\text{SFT}} = -\log \pi_{\theta}(y^* | x)$$

It teaches the model what a good assistant response looks like.

The limitation

- It imitates examples; it does not directly optimize a notion of response quality.
- If two answers are acceptable but one is better, chosen-only SFT does not use that comparison.
- It trains on demonstration data, not necessarily the model's own current failure modes.
- It can penalize valid alternatives if they differ from the written demonstration.

SFT is behavior cloning. RLHF adds preference optimization.

The Policy-Dependent Target

Human demonstration

A labeler answers using their own knowledge.

Q: population of Tuvalu?
A: around 11,000

If the model knows

SFT mostly teaches format: answer the question directly and fluently.

knowledge present -> good imitation

If the model does not know

SFT still pushes a confident target string, so generalization can become confident fabrication.

knowledge missing -> guess like the demo

For knowledge-seeking tasks, the right behavior depends on the model's epistemic state: answer if it knows, hedge or abstain if it does not. A fixed SFT target cannot encode that boundary.

Sources: John Schulman, "Reinforcement Learning from Human Feedback: Progress and Challenges," UC Berkeley EECS Colloquium, April 19, 2023; Yoav Goldberg, "Reinforcement Learning for Language Models," April 2023.

Part IV: From SFT to Preference Optimization

Why Post-Training Exists

Pretraining objective

$$-\log p_{\theta}(x_t \mid x_{<t})$$

Excellent for learning language, facts, code, and patterns from internet-scale text.

Assistant objective

- Answer the user directly.
- Use the requested format.
- Avoid unsafe or misleading behavior.
- Prefer helpful completions over unhelpful ones.

Many behaviors are easy for people to judge but hard to write as an exact differentiable reward.

Why Human Feedback?

Judge before you can specify

- Which answer is more helpful?
- Which summary is more faithful?
- Which refusal is appropriate?
- Which reasoning trace reaches the right conclusion?

RLHF starts from this asymmetry: comparing behavior is often easier than writing a reward.

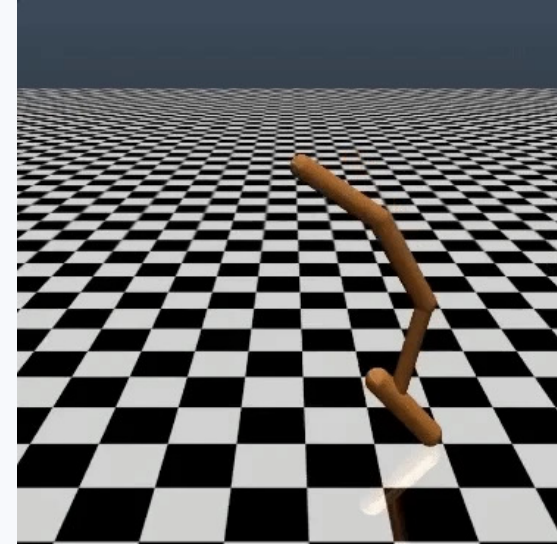
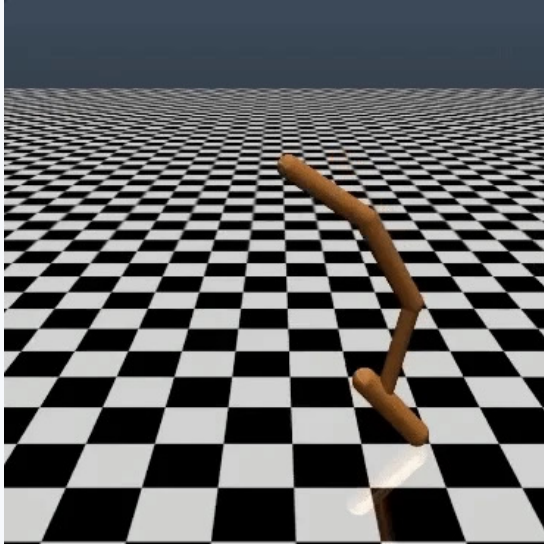
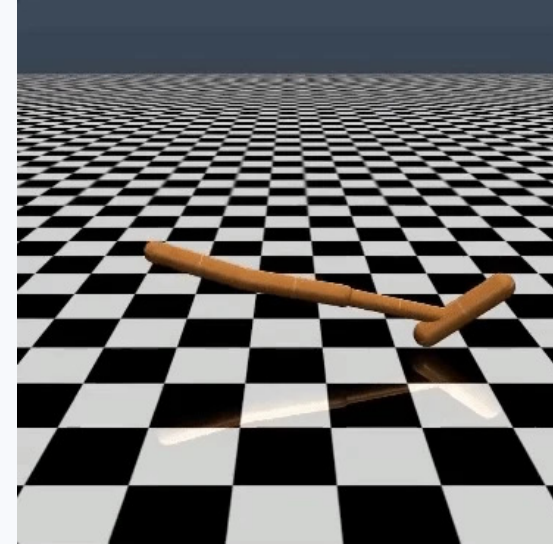


Figure source: Lambert SALA 2026 deck, based on Christiano et al., “Deep reinforcement learning from human preferences,” 2017.

Human Feedback vs. Hand-Written Rewards



Human judges compare behavior.



Hand-written rewards can miss what humans actually want.

RLHF says: learn a reward model from judgments, then optimize the policy against that learned model.

Figure source: Lambert SALA 2026 deck, based on Christiano et al., "Deep reinforcement learning from human preferences," 2017.

Base Model vs. Assistant

Base model behavior

Prompt: The president of the United States in 2006 was

Completion: George W. Bush, the governor of Florida in 2006 was Jeb Bush, and John McCain was an Arizona senator...

The model continues a plausible text stream.

Assistant behavior

User: The president of the United States in 2006 was

Assistant: George W. Bush was president in 2006. He served from January 20, 2001 to January 20, 2009.

The model answers the implied user request.

The historical shift is from completion quality to interaction quality.

Lecture 2 Guiding Questions

1. What behavior do we want?

Instruction following, helpfulness, safety, formatting, and multi-turn behavior.

2. How do we get labels?

Demonstrations and comparisons are easier than writing a full reward function.

3. What do we optimize?

SFT learns demonstrations. RLHF and DPO optimize preferences.

4. What can go wrong?

Reward hacking, over-optimization, labeler bias, and distribution shift.

5. Why not just use RL?

Language rewards are sparse, learned, noisy, and response-level.

6. How do we evaluate?

Once models become assistants, perplexity is no longer enough.

The Objective Shift: Likelihood to Reward

Generative modeling / SFT

The dataset gives the target tokens. Training asks the model to put probability mass on those tokens.

$$\min_{\theta} -\mathbb{E}_{(x,y^*) \sim \mathcal{D}} \log \pi_{\theta}(y^* | x)$$

Question: what text is likely under the training distribution?

RL / preference optimization

The model samples outputs, and a reward scores the behavior after generation.

$$\max_{\theta} \mathbb{E}_{y \sim \pi_{\theta}(\cdot|x)} [R(x, y)] - \beta D_{\text{KL}}(\pi_{\theta} || \pi_{\text{ref}})$$

Question: which behavior should this policy choose?

Signal

Likelihood gives dense token targets; RL gives response-level rewards.

Distribution

SFT trains on demonstrations; RL scores the model's own sampled behavior.

Failure mode

Likelihood can clone bad targets; RL can over-optimize a proxy reward.

Sources: Sutton and Barto, "Reinforcement Learning: An Introduction," 2018; Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," 2022.

The Three-Stage Recipe: Now RLHF

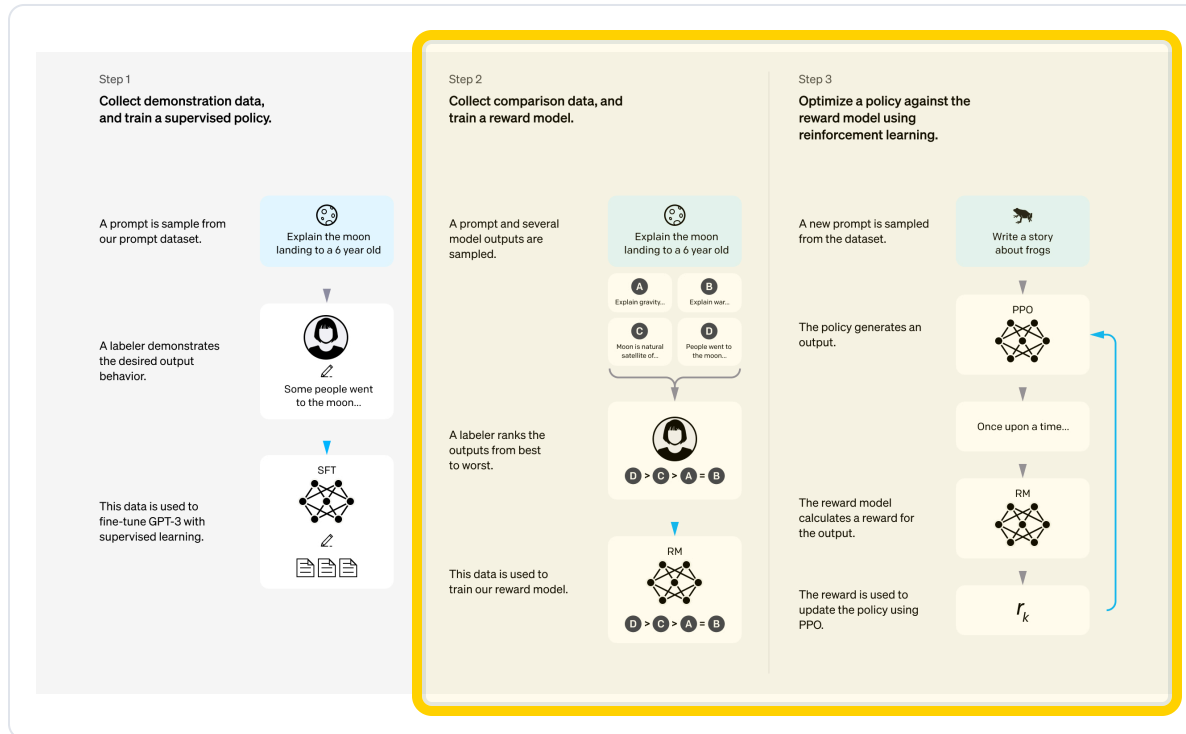


Figure source: Nathan Lambert SALA 2026 deck, adapted from Ouyang et al., "Training language models to follow instructions with human feedback," 2022.

1. Supervised fine-tuning

Start from an assistant policy that already follows the chat format.

2. Reward modeling

Sample candidate answers and learn a reward model from human rankings.

3. Policy optimization

Use the learned reward to update the policy, usually with a KL constraint to the SFT model.

RLHF begins once we stop asking only "what did the demonstrator write?" and start asking "which sampled response should this model prefer?"

RLHF Before Language Models

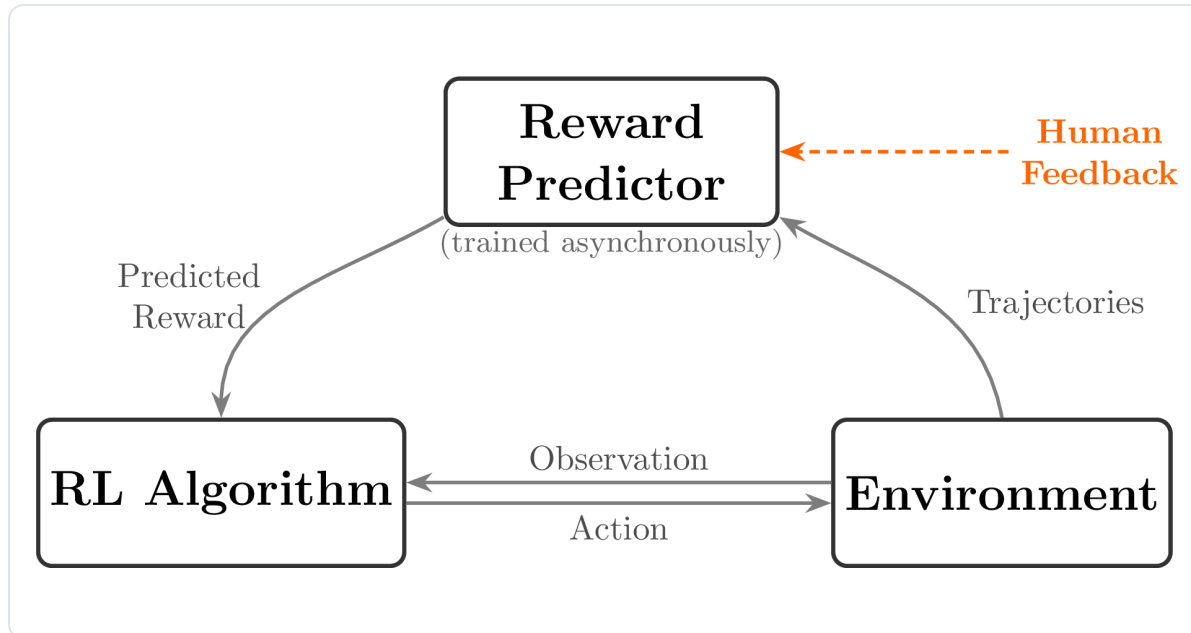


Figure source: Lambert SALA 2026 deck, based on Knox and Stone, "Interactively Shaping Agents via Human Reinforcement," 2008; Christiano et al., "Deep reinforcement learning from human preferences," 2017; Ziegler et al., "Fine-Tuning Language Models from Human Preferences," 2019.

Historical idea

- **TAMER**: learn from human feedback on actions.
- **Christiano et al. 2017**: learn rewards from trajectory preferences.
- **Ziegler et al. 2019**: apply preference learning to language models.
- **InstructGPT 2022**: make the three-stage language-model recipe canonical.

The Path to Modern RLHF

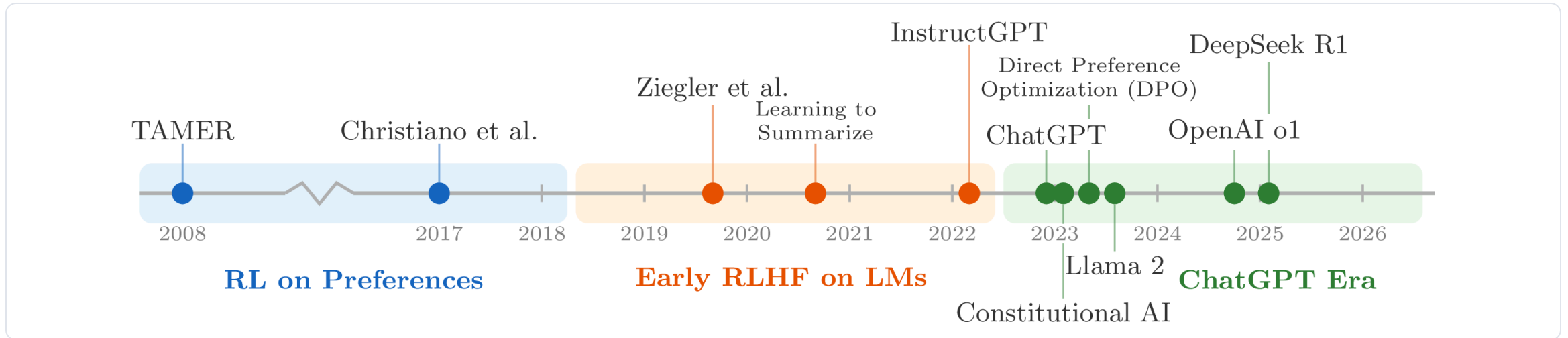


Figure source: Lambert SALA 2026 RLHF/post-training deck; timeline references include Knox and Stone 2008, Christiano et al. 2017, Ziegler et al. 2019, Ouyang et al. 2022, Rafailov et al. 2023.

The recurring pattern is: collect better behavioral data, choose an optimizer, and control how far the model moves.

When Did RL Enter the Language Pipeline?

2017

Human preferences train a reward model for RL agents in simulated control and Atari.

2019

Preference learning is applied to pretrained language models.

2020

Summarization models are optimized from human feedback and beat larger supervised baselines in human judgments.

2022

InstructGPT makes SFT, reward modeling, and PPO the canonical instruction-following recipe.

Sources: Christiano et al., “Deep reinforcement learning from human preferences,” 2017; Ziegler et al., “Fine-Tuning Language Models from Human Preferences,” 2019; Stiennon et al., “Learning to summarize from human feedback,” 2020; Ouyang et al., “Training language models to follow instructions with human feedback,” 2022.

Why RL?

Language generation as an action

For a prompt x , the model samples a response y . A reward model scores the whole response after it is produced.

$$y \sim \pi_{\theta}(\cdot \mid x), \quad r_{\phi}(x, y) \in \mathbb{R}$$

Optimize expected reward

RL lets us update the generator toward responses that receive higher reward, while limiting drift from the SFT/reference model.

$$\max_{\theta} \mathbb{E}_{y \sim \pi_{\theta}}[r_{\phi}(x, y)] - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$$

The reason for RL is not that SFT is bad. It is that once we have a scalar reward over sampled outputs, the natural objective is expected reward, not likelihood of one written answer.

Step 2: Preference Data

Instead of asking annotators to write the perfect answer, ask which of two answers is better.

chosen

Response A

Attention lets each token decide which earlier tokens to read from. It uses query-key similarity to compute weights, then averages value vectors and writes that information back into the residual stream.

Response B

Attention is when the model pays attention to words. It helps the model understand the sentence better and makes answers more accurate.

$(x, y_w, y_l) = \text{prompt, chosen response, rejected response}$

Rankings Become Pairs

Annotator task

For the same prompt, sample multiple candidate responses and ask labelers to rank or compare them.

Prompt x $\rightarrow$ responses $y_1, y_2, y_3, y_4 \rightarrow$ preference order

Rankings can be converted into many chosen/rejected pairs.

Why pairs are useful

- No absolute quality score is needed.
- They support a simple logistic likelihood.
- They expose subjective tradeoffs like concise versus complete.

$$(y_1 \succ y_3), (y_1 \succ y_4), (y_2 \succ y_3), \dots$$

Reward Model: A Scalar Head

Input

Concatenate the prompt and response, then run them through a transformer.

[prompt tokens] [response tokens]

Output

Replace the next-token head with a scalar reward head.

$$r_{\phi}(x, y) \in \mathbb{R}$$

The scalar is not truth. It is a learned prediction of preference.

Bradley-Terry tells us how to train scalar scores from pairwise judgments.

Bradley-Terry Reward Model

Turn preferences into probabilities

The reward model assigns a scalar score to each prompt-response pair.

$$r_{\phi}(x, y) \in \mathbb{R}$$

A response should win when it receives a higher reward score.

Bradley-Terry likelihood

PREFERENCE PROBABILITY

$$P(y_w \succ y_l \mid x) = \sigma(r_{\phi}(x, y_w) - r_{\phi}(x, y_l))$$

TRAINING LOSS

$$\mathcal{L}_{\text{RM}}(\phi) = -\log \sigma(r_{\phi}(x, y_w) - r_{\phi}(x, y_l))$$

Sources: Bradley and Terry, "Rank Analysis of Incomplete Block Designs: I. The Method of Paired Comparisons," 1952; Ouyang et al., "Training language models to follow instructions with human feedback," 2022.

Reward Models Are Proxy Objectives

They generalize imperfectly

The reward model only sees finite comparison data. Out-of-distribution responses can exploit blind spots.

They compress many values

Helpfulness, harmlessness, honesty, style, and instruction following are squeezed into one scalar.

They can be over-optimized

The response can get worse even while predicted reward improves.

This motivates the PPO/KL machinery: optimize preference, but keep the policy close enough to the model we trust.

Step 3: Optimize Against the Reward Model

Policy optimization loop

- Sample prompts x .
- Generate responses $y \sim \pi_{\theta}(\cdot | x)$.
- Score responses with $r_{\phi}(x, y)$.
- Update the policy with a KL penalty to stay near a reference model.

$$J(\pi) = \mathbb{E}[r_{\phi}(x, y)] - \beta D_{\text{KL}}(\pi || \pi_{\text{ref}})$$

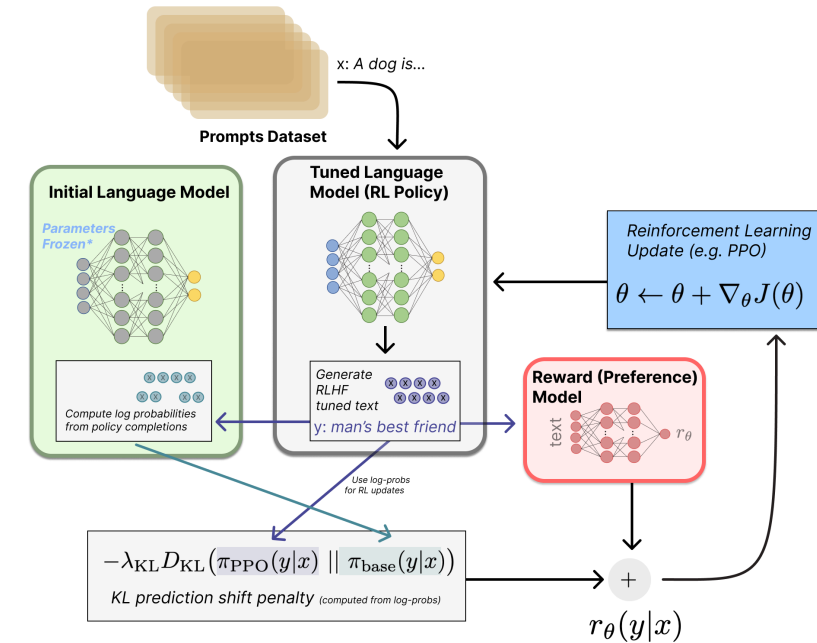


Figure source: Nathan Lambert SALA 2026 deck, RLHF overview slide; pipeline based on Ouyang et al., "Training language models to follow instructions with human feedback," 2022.

The KL Penalty Is the Guardrail

Why not maximize reward directly?

- The reward model is an imperfect proxy.
- Optimization can exploit proxy errors.
- Large policy shifts can degrade language quality or safety.

Reference-model anchoring

$$D_{\text{KL}}(\pi_{\theta}(\cdot \mid x) \parallel \pi_{\text{ref}}(\cdot \mid x))$$

The reference model keeps the optimized assistant close to the SFT model unless the reward improvement is worth the move.

PPO in RLHF: What Gets Updated?

One response is a trajectory

The prompt is fixed, the policy samples a whole response, and the reward usually arrives at the end.

$$y = (y_1, \dots, y_T), \quad \pi_\theta(y \mid x) = \prod_t \pi_\theta(y_t \mid x, y_{<t})$$

For LMs, RLHF is often closer to contextual bandit optimization than long-horizon robotics RL.

PPO uses an old policy as a trust region

PROBABILITY RATIO

$$\rho_t(\theta) = \frac{\pi_\theta(y_t \mid h_t)}{\pi_{\text{old}}(y_t \mid h_t)}$$

CLIPPED OBJECTIVE

$$\mathcal{L}_{\text{clip}} = \mathbb{E} [\min(\rho_t A_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) A_t)]$$

Source: Schulman et al., “Proximal Policy Optimization Algorithms,” 2017; Ouyang et al., “Training language models to follow instructions with human feedback,” 2022.

PPO in RLHF: Reward, KL, Advantage

Reward

The learned reward model scores the completed response.

$$r_{\phi}(x, y)$$

KL penalty

Subtract a cost for moving away from the reference model.

$$-\beta \log \frac{\pi_{\theta}(y \mid x)}{\pi_{\text{ref}}(y \mid x)}$$

Advantage

Estimate whether this response was better than expected.

$$A \approx R - V(x)$$

Why PPO was used

- Works with sampled text, not just fixed labels.
- Optimizes a learned scalar reward directly.
- Controls update size with clipping and KL.

Why it is delicate

- Expensive: generate, score, and update repeatedly.
- Sensitive to reward-model errors.
- Needs careful normalization and value modeling.

From PPO to DPO

PPO route

- Train a reward model from comparisons.
- Sample new responses from the policy.
- Use PPO to optimize reward minus KL to the reference model.

$$\max_{\pi} \mathbb{E}_{y \sim \pi} [r(x, y)] - \beta D_{\text{KL}}(\pi \| \pi_{\text{ref}})$$

DPO question

- The data already says y_w should beat y_l .
- KL-regularized RLHF links rewards to policy/reference log-ratios.
- Can we train that preference directly, without an explicit reward model?

$$r(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x)$$

DPO is not chosen-only SFT. It uses both chosen and rejected responses, and keeps the reference model inside the loss.

Sources: Ouyang et al., “Training language models to follow instructions with human feedback,” 2022; Rafailov et al., “Direct Preference Optimization,” 2023.

DPO: Eliminate the Explicit Reward Model

Start from KL-regularized RLHF

$$\max_{\pi} \mathbb{E}_{y \sim \pi(\cdot | x)} [r(x, y)] - \beta D_{\text{KL}}(\pi(\cdot | x) \| \pi_{\text{ref}}(\cdot | x))$$

The optimal policy has a closed-form relationship to the reward.

Reward as a log-ratio

$$\pi^*(y | x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y | x) \exp(r(x, y)/\beta)$$

$$r(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x)$$

In a pairwise comparison, the unknown $\log Z(x)$ cancels.

Source: Rafailov et al., "Direct Preference Optimization: Your Language Model is Secretly a Reward Model," 2023.

DPO Loss: Bradley-Terry With Policy Ratios

Define an implicit reward score

$$s_{\theta}(x, y) = \beta [\log \pi_{\theta}(y \mid x) - \log \pi_{\text{ref}}(y \mid x)]$$

Chosen responses should have larger implicit reward than rejected responses.

Pairwise objective

$$\mathcal{L}_{\text{DPO}}(\theta) = -\log \sigma(s_{\theta}(x, y_w) - s_{\theta}(x, y_l))$$

- Increase chosen completions relative to the reference.
- Decrease rejected completions relative to the reference.
- β controls movement away from the reference.

Source: Rafailov et al., “Direct Preference Optimization: Your Language Model is Secretly a Reward Model,” 2023.

PPO vs. DPO

Axis	PPO-style RLHF	DPO-style preference optimization
Training signal	Reward model scores sampled responses.	Chosen/rejected pairs directly.
Generation during training	Online rollouts from the current policy.	Usually offline preference dataset.
Extra models	Reward model, value model, reference model.	Reference model; implicit reward via log-ratios.
Main control knob	KL coefficient, PPO clip, reward normalization.	β against the reference policy.
Failure modes	Reward hacking, instability, high cost.	Data quality limits, overfitting pairwise preferences.

Modern post-training often mixes supervised data, preference data, direct objectives, RL on verifiable tasks, and filtering/evaluation loops.

Part V: Reasoning Models

The o1 Moment

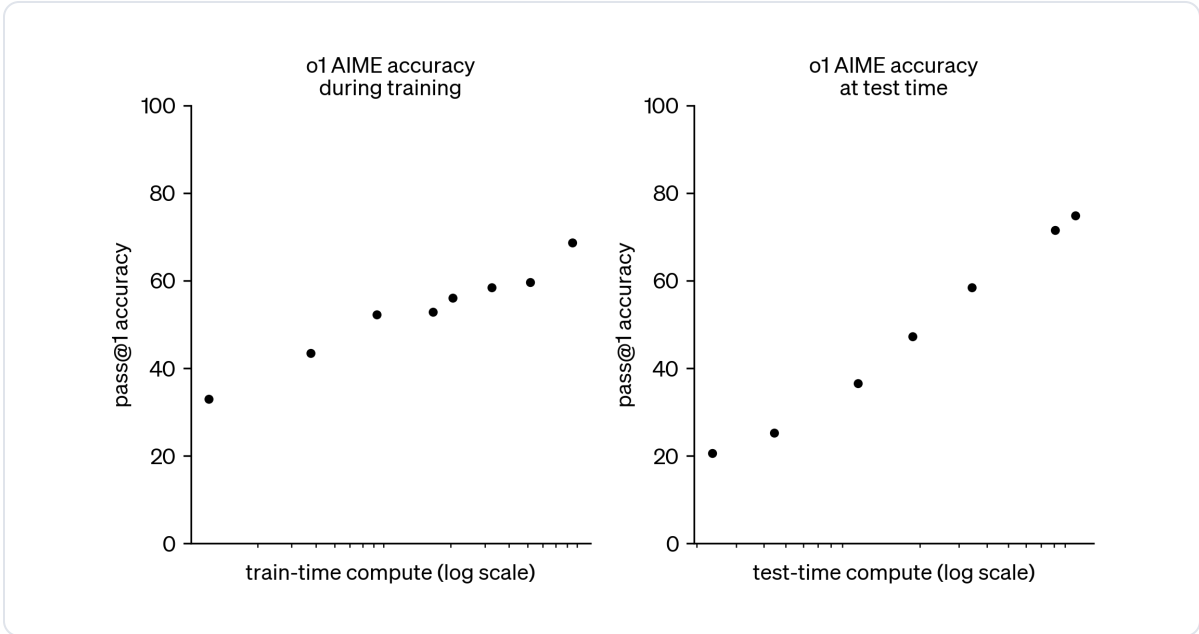


Figure source: OpenAI, “Learning to reason with LLMs,” 2024. The plots show o1 AIME pass@1 accuracy versus RL training compute and test-time compute on log-scale axes.

What changed publicly?

On September 12, 2024, OpenAI introduced o1 as a model trained to spend more time reasoning before answering.

- Large-scale RL teaches productive chain-of-thought behavior.
- Performance improves with both RL training compute and test-time thinking compute.
- The x-axis units were not disclosed; the public claim is the new scaling direction.

Eval	GPT-4o	o1-preview	o1
AIME 2024 pass@1	9.3	44.6	74.4
AIME 2024 cons@64	13.4	56.7	83.3
Codeforces Elo / pct.	808 / 11th	1,258 / 62nd	1,673 / 89th
GPQA Diamond pass@1	50.6	73.3	77.3

Source: OpenAI, “Learning to reason with LLMs,” 2024, compute scaling figure and Appendix A. AIME and GPQA values are percentages; Codeforces reports Elo and percentile.

The Speculation Gap

The Suspects

- Guess + Check
- Process Rewards
- Search / AlphaZero
- Learning to Correct

Public clues, hidden recipe

- OpenAI showed the behavior and scaling curves.
- The exact o1 training and inference recipe was not public.
- The community asked: is this verifier-guided sampling, process rewards, tree search, self-correction, or some mixture?

This is the right historical moment to separate the phenomenon from the implementation.

Figure source: Sasha Rush and Daniel Ritter, “Speculations on Test-Time Scaling (o1),” tutorial slides, 2024.

Were the Suspects Right?

Direction	What later public recipes suggest
Guess + Check	Clearly useful: sampling, majority vote, reranking, and rejection sampling all became standard ways to spend inference or data-generation compute.
Process Rewards	Partly: process reward models are important, but DeepSeek-R1’s public recipe emphasized outcome and format rewards rather than an explicit process-reward model.
Search / AlphaZero	Unresolved for o1. DeepSeek-R1 did not require public tree search, though verifier-guided search remains important in math/code systems.
Learning to Correct	Strongly aligned: o1 descriptions and R1 behaviors emphasize checking, revising, and trying alternate strategies.

Careful answer: these directions were not “wrong.” DeepSeek showed one open path: RL on verifiable reasoning tasks can induce useful reasoning behavior without needing the exact closed o1 recipe.

Sources: Rush and Ritter, “Speculations on Test-Time Scaling (o1),” 2024; OpenAI, “Learning to reason with LLMs,” 2024; DeepSeek-AI et al., “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,” 2025.

DeepSeek-R1: The Recipe Becomes Visible

R1-Zero

Start from a base model and apply RL directly on verifiable reasoning tasks.

Emergence

Longer reasoning, self-reflection, verification, and strategy switching appear through RL.

Problems

Pure RL can produce poor readability, repetition, and language mixing.

R1

Add cold-start data, reasoning RL, rejection sampling/SFT, and final RL.

The key lesson is not that every reasoning model must copy R1. It is that verifiable rewards make reasoning optimization much less mysterious.

Source: DeepSeek-AI et al., “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,” 2025, arXiv:2501.12948.

What Data Trains Reasoning Behavior?

Math prompt + rule reward

Prompt: contest-style problem with a deterministic final answer.

Verifier: exact answer check plus required tags.

```
<think>
Convert the expression into a geometry/optimization problem.
Derive a compact formula for  $S_n$ .
Check the integer condition by factoring  $m^2 - n^4$ .
</think>
<answer>12</answer>
```

Code prompt + executable reward

Prompt: implement a small function/class from a spec.

Verifier: compile and run public/hidden tests.

```
<think>
Map requirements to a data structure.
Check the missing-key branch against the example.
Return code that passes the unit tests.
</think>
<answer>class Dictionary: ...</answer>
```

Reasoning RL does not need humans to grade every step. It needs tasks where the final answer can be checked and a format reward can keep the trace parseable.

Sources: DeepSeek-AI et al., “DeepSeek-R1,” 2025, arXiv:2501.12948, R1-Zero template and reward design; DeepSeek-R1 arXiv source sample files `data/sft_sample_math.md` and `data/sft_sample_code.md`. Traces shown here are shortened teaching examples.

DeepSeek-R1-Zero: The “Aha Moment”

DeepSeek-R1-Zero “aha moment” excerpt

The paper reports an intermediate model rethinking its own derivation inside the reasoning trace.

Question

If $a > 1$, solve a nested-radical equation and find the sum of the real solutions.

Response: `<think>`

Square the equation, isolate the inner radical, expand ...

Check whether the algebraic transformation preserved constraints ...

Wait, wait. Wait. That's an aha moment I can flag here.

Re-evaluate step by step, then continue toward the final answer ...

Source: DeepSeek-AI et al., “DeepSeek-R1,” 2025, arXiv source, Table “aha moment”. Text is abbreviated for slide legibility.

What to notice

- The trace is inside `<think>...</think>` ; the answer is separated from the reasoning process.
- The model begins to revise its own derivation instead of only sampling a final answer.
- This is why test-time compute becomes meaningful: the extra tokens can be used for checking, backtracking, and strategy changes.

Figure source: DeepSeek-AI et al., “DeepSeek-R1,” 2025, arXiv source, Table “aha moment”; abbreviated for slide legibility.

Source: DeepSeek-AI et al., “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,” 2025, arXiv:2501.12948.

Preference Rewards vs. Verifiable Rewards

Preference reward

Used when the target is subjective or hard to specify exactly.

Which answer is more helpful?

- Good for style, helpfulness, safety, and conversation quality.
- Requires human or AI comparisons.
- Reward model is a proxy.

Verifiable reward

Used when correctness can be checked automatically.

$$R(x, y) = \mathbf{1}\{\text{verify}(x, y) = \text{correct}\}$$

- Natural for math, code, formal proofs, and tool-checked tasks.
- Reward can be sparse but less subjective.
- Supports RL over reasoning attempts.

Reasoning Effort: A New Knob

Think longer

Generate more internal reasoning tokens before committing to the final answer.

Sample more

Generate multiple candidate solutions and use voting, scoring, or verification.

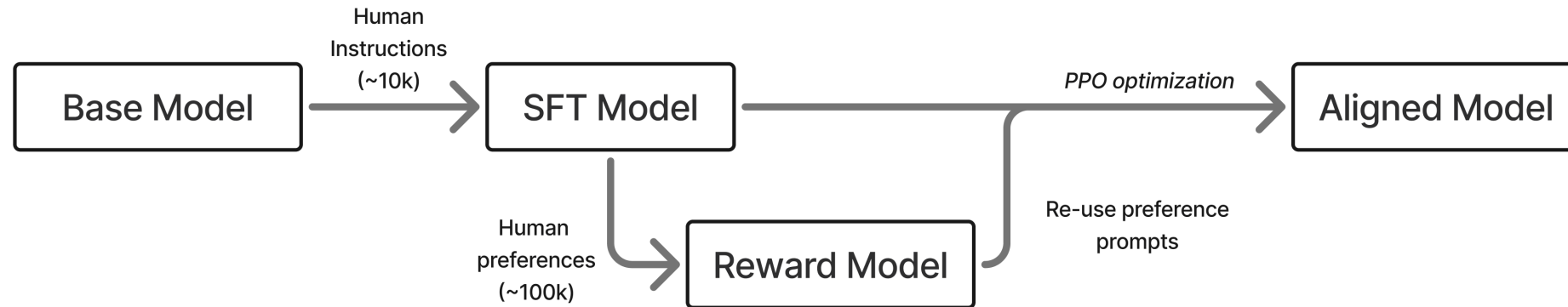
Verify more

Use tests, answer checkers, learned verifiers, or tool calls to select better outputs.

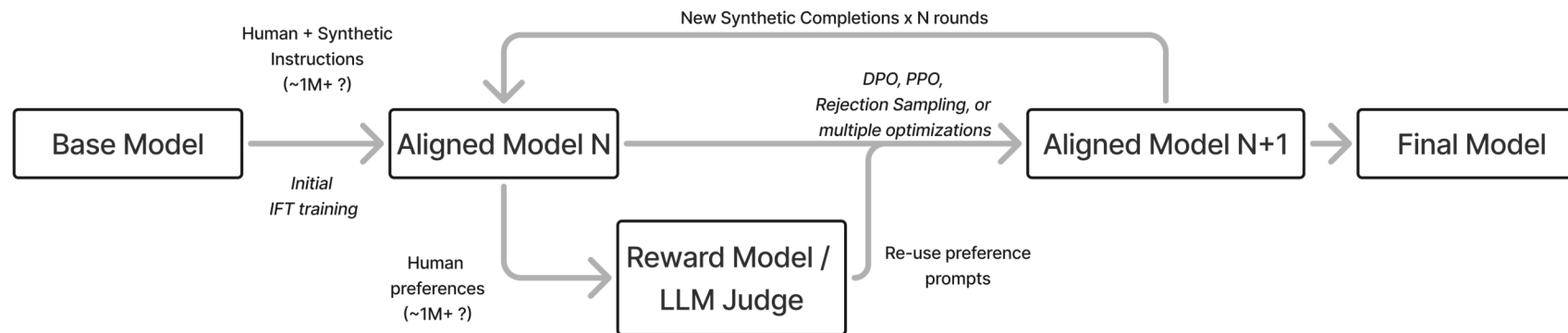
Reasoning effort is inference-time scaling. It can help dramatically on verifiable hard problems, but more tokens alone are not a guarantee of correctness.

Sources: OpenAI, “Learning to reason with LLMs,” 2024; DeepSeek-AI et al., “DeepSeek-R1,” 2025; Zeng et al., “Revisiting the Test-Time Scaling of o1-like Models,” 2025.

From RLHF to Post-Training



Early recipe: supervised fine-tuning, reward modeling, RL.



Modern recipe: staged optimizers, filters, datasets, and evaluations. Figure source: Nathan Lambert SALA 2026 deck.

Figure source: Nathan Lambert, SALA 2026 RLHF/post-training deck.

Modern Post-Training Stack

SFT data

Demonstrations, chat formatting, domain adaptation, safety behavior.

Preference data

Human or AI comparisons between responses for the same prompt.

Optimization

PPO, DPO, rejection sampling, RLVR, reasoning RL, or mixtures of objectives.

Evaluation loop

Benchmarks, model judges, human evals, safety evals, and regression tests.

The word “RLHF” now often refers to one stage inside a broader post-training pipeline.

Part VI: Evaluation

Why Perplexity Is Not Enough

Perplexity answers

Did the model assign high probability to held-out text?

This is still useful for pretraining and regression checks.

Assistant evaluation asks

- Did it follow the instruction?
- Was it useful and truthful?
- Was it safe?
- Was it robust across prompt formats?

Post-training changes behavior, so evaluation must measure behavior.

What Do We Evaluate?

Capability

Question answering, coding, math, reasoning, tool use, long-context tasks.

Preference

Which response do humans or judges prefer for the same prompt?

Reliability

Factuality, calibration, robustness, and consistency under perturbations.

Safety

Refusal quality, harmful content, privacy leakage, jailbreak robustness.

Example benchmark families: MMLU, BIG-bench, HELM, MT-Bench, Chatbot Arena. Exact comparisons depend on prompt format, decoding, data, and protocol.

Benchmark Snapshot: Open Reasoning Model

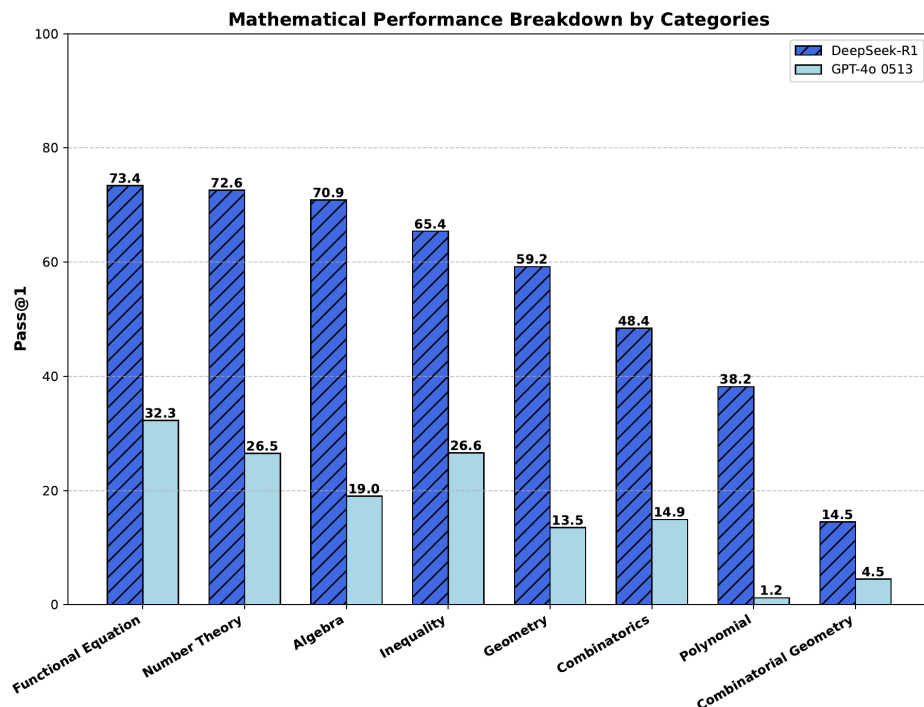


Figure source: DeepSeek-AI et al., “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,” 2025, arXiv source, [figures/math_performance_breakdown_w_baseline.pdf](#).

Source: DeepSeek-AI et al., “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,” 2025, arXiv:2501.12948. The authors report pass@1 scores by mathematical category and use GPT-4o 0513 as the baseline.

What this kind of benchmark tells us

- A single aggregate score hides which subskills improved.
- Here, the open reasoning model separates strongly from GPT-4o on competition-math categories.
- The y-axis is pass@1: one answer attempt, no majority vote.
- Benchmarking is most useful when the task family matches the capability we care about.

How to Read Benchmark Tables

Question	Why it matters
Prompting	Zero-shot, few-shot, chain-of-thought, and formatting can change scores.
Decoding	Temperature, best-of-N, and tool access can change outputs.
Contamination	The benchmark may have appeared in pretraining or post-training data.
Saturation	Easy benchmarks stop separating strong models.
Judging	Human and model judges have biases and variance.
Distribution shift	A benchmark may not match real deployment.

Evaluation After Post-Training

Offline automatic evals

Fast, cheap, repeatable. Good for broad regression testing.

Human or model preference evals

Closer to user experience, but noisier and more expensive.

Red-team and safety evals

Stress-test failure modes that average benchmark scores hide.

Evaluation is not a final slide after training. It is part of the post-training loop.

What We Covered Today

- Scaling laws made base-model pretraining forecastable.
- Chinchilla showed that compute-optimal training needs enough data, not only more parameters.
- ChatGPT-style behavior requires post-training data and objectives.
- RLHF turns human comparisons into reward models and then policy updates.
- DPO trains on the same preference signal more directly.
- o1 made test-time compute visible; DeepSeek-R1 made a public reasoning-RL recipe concrete.
- Evaluation changes once the model is an assistant rather than a completion engine.

Next lecture: modern systems and architectures: RoPE, RMSNorm, SwiGLU, KV cache, MQA/GQA, FlashAttention, MoE, and serving constraints.